

Shared State in EDSL

Foundations for coordinated multi-agent studies

Expected Parrot

August 2026

Contents

1	Why shared state exists	3
2	Complete example: an open activity poll	4
3	The core objects	6
4	SharedState and SharedStateMap	7
4.1	A state containing one primitive	7
4.2	Why votes use a register	7
4.3	Putting the state in a map	8
4.4	Homogeneous values	8
4.5	The map after a run	9
5	Survey steps and runtime values	10
5.1	Writes do not fail an interview	10
5.2	Selecting which state receives the write	11
5.3	Deferred runtime references	11
5.4	Explicit answer references	12
6	Writing dynamic question text with Stem	13
7	Reading and refreshing state	14
8	Binding the map to state keys	16
8.1	One fixed state	16
8.2	One state per team	16
8.3	Keys	17
9	State goes into and comes out of a run	18
9.1	Continuing with the output of an earlier run	18
9.2	Local and remote execution	19
10	Schedules are typed objects	20
10.1	Serial participation	20
10.2	Concurrent independent interviews	20
10.3	Simultaneous rounds	20
10.4	Ordered participation within concurrent teams	21
11	Extension: one poll per group	22

12 Primitive reference and question composition	24
12.1 General coordination primitives	24
12.2 Allocation, matching, and market primitives	26
12.3 Experimental-game primitives	26
12.4 Configured primitives	27
12.5 Compatibility is checked at creation	28
13 Inspecting a completed run	29
14 Closing states during execution	31
15 Serialization and remote execution	32
16 Validation before execution	33
17 Mental checklist	34

Chapter 1

Why shared state exists

An ordinary EDSL survey interview is self-contained. An agent answers questions, and EDSL records those answers in **Results**. This works when one respondent's interview should not affect another's.

Some studies require coordination. Participants may contribute to a common discussion, claim different assignments, revise a shared document, or work in separate teams. In these studies, one interview must be able to change information that another interview can later read.

EDSL calls this information **shared state**.

Shared state does not replace surveys or results. It creates a second data path:

```
Agent interview -- answers -----> Results
      |
      +--- typed operations --> shared state --> later interviews
```

The two paths serve different purposes:

- **Results** preserve model answers, prompts, and interview metadata.
- Shared state preserves coordinated actions and the evolving state of the study.

Chapter 2

Complete example: an open activity poll

Four people vote for a weekend activity. Each person sees the votes already submitted before choosing.

```
from edsl import Agent, AgentList, Model
from edsl import QuestionMultipleChoice, Stem, Survey, current
from edsl.schedules import Serial
from edsl.sharedstate import (
    SharedRegister,
    SharedState,
    SharedStateMap,
)

ACTIVITIES = (
    "bike ride",
    "sailing",
    "hike",
    "beach day",
)

people = AgentList([
    Agent(name="Amina"),
    Agent(name="Boris"),
    Agent(name="Chen"),
    Agent(name="Daria"),
])

poll = SharedState(
    votes=SharedRegister(
        value_options=ACTIVITIES,
        write_once=True,
    ),
)

activity_polls = SharedStateMap({
```

```

    "weekend-activity": poll,
  })

  activity = QuestionMultipleChoice(
    question_name="activity",
    question_text=Stem(
      "Votes submitted so far:\n{votes}\n\n"
      "Which activity do you prefer?",
      votes=current.state.votes,
    ),
    question_options=ACTIVITIES,
  )

  survey = Survey([
    activity,
    poll.votes.set(
      key=current.agent.name,
      value=activity.answer,
    ),
  ])

  results = (
    survey
    .by(people)
    .by(Model("gemini-2.5-flash"))
    .run(
      shared_state=activity_polls.at("weekend-activity"),
      interview_schedule=Serial(),
      cache=False,
    )
  )

```

The run realizes one map entry:

```

activity_polls
+-- "weekend-activity" -> SharedState
   +-- "votes" -> SharedRegister

```

Amina sees no earlier votes. Boris sees Amina's vote. Chen sees the first two, and Daria sees the first three. This is therefore an **open sequential poll**, not a secret ballot. Order and social influence are features of the study.

The chapters that follow unpack this program one concept at a time: the state, the map, survey write steps, runtime references, routing, snapshots, and schedule.

Chapter 3

The core objects

The shared-state interface separates one primitive container from a keyed collection of those containers. A completed run records the collection both as it entered and as it left the run.

Object	When used	Responsibility
<code>SharedState</code>	Before, during, and after a run	Contains named primitives
<code>SharedStateMap</code>	Before, during, and after a run	Maps scope keys to <code>SharedState</code> values
<code>Results.shared_state</code>	After a run	Contains immutable before and after snapshots

The lifecycle is:

```
SharedState(votes=SharedRegister(...))
  |
  +-- SharedStateMap({"weekend-activity": state})
      |
      +-- run
          |
          +-- results.shared_state.before
          +-- results.shared_state.after
```

This separation is deliberate:

- A `SharedState` owns primitives, not a key or store.
- A map owns scope keys, not primitive definitions.
- A run may use internal coordination infrastructure, but it is not part of the study-facing API.
- Every map value is a `SharedState`.

Chapter 4

SharedState and SharedStateMap

4.1 A state containing one primitive

A `SharedState` is a named collection of primitives:

```
from edsl.sharedstate import SharedRegister, SharedState

ACTIVITIES = (
    "bike ride",
    "sailing",
    "hike",
    "beach day",
)

poll = SharedState(
    votes=SharedRegister(
        value_options=ACTIVITIES,
        write_once=True,
    ),
)
```

`votes` is a name chosen by the researcher. `SharedRegister(...)` is the primitive stored under that name. The state says:

This shared state contains one register named `votes`. Its keys are voter names, its values must be one of the four activities, and each key can be written once.

At this point no event has been written and there is nothing to read. The object contains the primitive configuration and its initial value, independent of where inference will execute.

4.2 Why votes use a register

The current value of `votes` is dictionary-like:

```
{}
```

```
{"Amina": "sailing"}
```

```
{"Amina": "sailing", "Boris": "hike"}
```

Each voter name is a key and each selected activity is its value. `SharedRegister` provides the poll's required behavior:

- `write_once=True` preserves the first value submitted for each voter;
- `value_options=ACTIVITIES` prevents an out-of-ballot value from becoming the register's value;
- looking up a voter has ordinary dictionary semantics;
- the current state contains one authoritative vote per person.

The run still records every processed `set` operation as an event. The study keeps an audit history without making its reader-facing state an event log.

4.3 Putting the state in a map

A `SharedStateMap` is an ordinary keyed container whose values are `SharedState` objects:

```
from edsl.sharedstate import SharedStateMap

activity_polls = SharedStateMap({
    "weekend-activity": poll,
})
```

The complete hierarchy is visible in the code:

```
activity_polls: SharedStateMap
+-- "weekend-activity": SharedState
    +-- votes: SharedRegister
```

The string `"weekend-activity"` is the **scope key**. A scope is not a property hidden inside `SharedState`; it is one entry in the surrounding map.

Before execution, the author may add or replace entries using normal mapping syntax:

```
activity_polls["weekday-activity"] = SharedState(
    votes=SharedRegister(
        value_options=ACTIVITIES,
        write_once=True,
    ),
)
```

Preflight freezes the map. Keys cannot be added, removed, or replaced while a run is in progress.

4.4 Homogeneous values

All states in one map have the same primitive names and types. This lets one survey operate safely against any selected entry:

```
family_polls = SharedStateMap({
    "family-a": SharedState(
        votes=SharedRegister(value_options=ACTIVITIES, write_once=True),
    ),
})
```

```
"family-b": SharedState(  
    votes=SharedRegister(value_options=ACTIVITIES, write_once=True),  
),  
})
```

Map construction rejects incompatible values:

```
StateSchemaError: primitive 'votes' in map value 'family-b' is incompatible;  
expected SharedRegister(value_options=ACTIVITIES, write_once=True).
```

4.5 The map after a run

The completed run exposes immutable snapshots of the map:

```
before = results.shared_state.before  
after = results.shared_state.after  
  
after.keys()  
after.values()  
after.items()  
len(after)  
after["weekend-activity"]
```

`before` is an immutable copy of the complete map referenced by the supplied binding. `after` is an immutable copy of that map when the run completed. The result records the binding separately:

```
results.shared_state.binding
```

Keeping all three makes the state transition explicit and lets a saved **Results** object reproduce which map entries each interview observed and what the run produced.

Chapter 5

Survey steps and runtime values

A survey is an ordered sequence of executable steps. Most steps are questions; shared-state operations are write steps placed among those questions:

```
survey = Survey([
    activity,
    poll.votes.set(
        key=current.agent.name,
        value=activity.answer,
    ),
])
```

`activity` asks the question. `poll.votes.set(...)` constructs a declarative shared-state write step. Its position immediately after `activity` says when the write occurs: after the answer has been produced and validated.

The write step does not modify state while the survey is being constructed. At interview time, EDSL executes the survey in order:

1. Render the `activity` question using the selected state's current view.
2. Obtain and validate the answer.
3. Resolve `current.agent.name` and `activity.answer`.
4. Execute `votes.set(...)` against the selected state.
5. Continue to the next survey step.

The answer remains in `Results`, and the `set` operation also records the vote in shared state. Because the register is `write_once`, the same voter cannot submit a second value.

The ordering is part of the study definition. A write can affect the shared state seen by later questions and later interviews.

5.1 Writes do not fail an interview

A shared-state write step always succeeds from the survey's point of view. After EDSL submits the operation, the interview continues. A conflict, duplicate key, retry, or concurrent write cannot turn the write step into a failed question, trigger branching, or require the agent to answer again.

The primitive still enforces its declared transition rules. For example, `write_once=True` preserves the first value for a key, so a later `set` for the same key may leave the register unchanged. “The write

succeeds” means that the operation was processed without disrupting the interview; it does not promise that the requested value became the current value.

Any value returned by a write operation is **advisory only**. Code must not use it as an acknowledgement, as proof that a particular value won a race, or as a guaranteed read-after-write view. In particular, do not write survey logic such as:

```
receipt = poll.votes.set(
    key=current.agent.name,
    value=activity.answer,
)

# Invalid design: a write has no promised authoritative return value.
if receipt.value == activity.answer:
    ...
```

When a later step needs authoritative state, it reads `current.state` under the visibility rules declared by the interview schedule. The **after** snapshot in **Results** is the authoritative record of the completed run.

This runtime rule does not weaken construction-time checks. Invalid primitive definitions, unresolved references, incompatible map entries, and impossible routing still fail preflight before any interview begins.

5.2 Selecting which state receives the write

The survey step identifies the primitive and operation. The run’s state binding identifies which `SharedStateMap` entry receives it.

A fixed binding sends every interview to one scope key:

```
shared_state=activity_polls.at("weekend-activity")
```

A dynamic binding resolves a key separately for each interview:

```
shared_state=family_polls.by(
    current.assignment.family_groups.group_id
)
```

Every resolved key must already exist in the map. The binding controls the space in which the entire interview reads and writes; individual write steps do not choose their own scope keys.

5.3 Deferred runtime references

The study is constructed before an interview runs. There is therefore no active agent, answer, assignment, round, or state while the Python objects are being created.

EDSL represents those eventual values with the `current` namespace:

```
from edsl import current

current.agent.name
current.agent.department
current.assignment.family_groups.group_id
```

```
current.assignment.family_groups.position
current.run.round
current.state.votes
```

These expressions do not evaluate immediately. Each creates a checked, serializable reference that resolves against the active interview.

For example:

```
poll.votes.set(
    key=current.agent.name,
    value=activity.answer,
)
```

The arguments have three different sources:

Expression	Runtime source
<code>current.agent.name</code>	The active agent
<code>activity.answer</code>	An earlier question's answer

A literal remains a literal:

```
poll.votes.set(
    key="facilitator",
    value="sailing",
)
```

Here both strings are stored unchanged. Neither can be confused with a runtime reference.

5.4 Explicit answer references

Every question exposes `.answer`:

```
activity = QuestionMultipleChoice(
    question_name="activity",
    question_text="Which activity do you prefer?",
    question_options=ACTIVITIES,
)

write_vote = poll.votes.set(
    key=current.agent.name,
    value=activity.answer,
)
```

`activity.answer` is not an answer yet. It means:

When this operation executes, use the active interview's answer to the question named `activity`.

Survey construction verifies that `activity` appears before `write_vote` and that its answer type is compatible with the primitive operation.

Chapter 6

Writing dynamic question text with Stem

Use an ordinary string when `question_text` is static:

```
QuestionMultipleChoice(  
    question_name="activity",  
    question_text="Which activity do you prefer?",  
    question_options=ACTIVITIES,  
)
```

Use `Stem` whenever the text includes runtime information:

```
from edsl import QuestionMultipleChoice, Stem, current  
  
activity = QuestionMultipleChoice(  
    question_name="activity",  
    question_text=Stem(  
        "Votes submitted so far:\n{votes}\n\n"  
        "You are {name}. Which activity do you prefer?",  
        votes=current.state.votes,  
        name=current.agent.name,  
    ),  
    question_options=ACTIVITIES,  
)
```

The prose uses named placeholders. Python expressions supply their values. `Stem` checks that every placeholder is bound and records which state fields the question will read.

Because state reads are explicit, EDSL can validate visibility before making model calls. `Stem` compiles to a serializable Jinja template. A worker renders that template against the interview context available at that point in the survey.

Chapter 7

Reading and refreshing state

An interview begins with one implicit state read. EDSL loads one atomic snapshot of the bound `SharedState` into `current.state`. Questions render from that snapshot until the survey explicitly refreshes it.

Writes do not refresh `current.state`. Their returned values are advisory, so a write cannot establish what a later question should regard as authoritative. Use the declared state's `.refresh()` method when later questions should see a newly authorized view:

```
reaction = QuestionFreeText(  
    question_name="reaction",  
    question_text=Stem(  
        "The current votes are:\n{votes}\n\n"  
        "What do you think of the group's choices?",  
        votes=current.state.votes,  
    ),  
)  
  
survey = Survey([  
    activity,  
    poll.votes.set(  
        key=current.agent.name,  
        value=activity.answer,  
    ),  
    poll.refresh(),  
    reaction,  
])
```

The sequence is:

implicit initial read	-> <code>current.state</code> contains the initial authorized snapshot
<code>activity</code>	-> renders from that snapshot
<code>votes.set</code>	-> submits a write; <code>current.state</code> does not change
<code>poll.refresh()</code>	-> replaces <code>current.state</code> with a newly authorized snapshot
<code>reaction</code>	-> renders from the refreshed snapshot

`poll.refresh()` refreshes the entire bound poll state atomically. `poll` identifies the state declaration; the run's binding identifies the concrete map entry for the active interview. The schedule decides which

version the refresh may return. Under `Serial()`, this is normally the latest completed state. During a simultaneous round, a refresh may still return the round-start snapshot until the schedule reveals that round's writes.

`.refresh()` constructs a survey step, not an immediate read. Internally that step serializes as a typed `StateRead` operation. It makes no model call and has no agent answer. A failed refresh cannot silently retain stale state and continue; EDSL retries it according to execution policy or fails the interview. A model-provider retry does not repeat the read that supplied an already-rendered question.

The method name distinguishes survey authoring from immediate inspection:

```
poll.refresh()  # construct a deferred survey step
poll.read()     # immediately inspect a concrete realized state
```

Inspection expands the implicit initial read, so the executed survey plan shows every point at which `current.state` could change.

Chapter 8

Binding the map to state keys

A map contains the available states. A binding defines which one an interview receives.

8.1 One fixed state

Use `.at(key)` when every interview shares one world:

```
weekend_poll = activity_polls.at("weekend-activity")
```

This binding means:

For every interview in this run, bind the state whose key is `weekend-activity` as `current.state`.

It does not create storage and does not write an event.

8.2 One state per team

An `AgentGrouping` associates each agent with a group and an ordered position inside that group:

```
from edsl.assigments import AgentGrouping

family_groups = AgentGrouping(
    name="family_groups",
    groups={
        "family-a": ["Amina", "Boris"],
        "family-b": ["Chen", "Daria"],
    },
)

participants = people.with_assignments(family_groups)
```

The list order supplies each member's position. `with_assignments(...)` joins the grouping to the `AgentList` by stable agent identity. Persona traits remain unchanged.

Use `.by(reference)` to route each assigned participant to a map key:

```

family_polls = SharedStateMap({
  "family-a": SharedState(
    votes=SharedRegister(value_options=ACTIVITIES, write_once=True),
  ),
  "family-b": SharedState(
    votes=SharedRegister(value_options=ACTIVITIES, write_once=True),
  ),
})

group_polls = family_polls.by(
  current.assignment.family_groups.group_id
)

```

Suppose four participants have assignments:

Participant	group_id
Amina	family-a
Boris	family-a
Chen	family-b
Daria	family-b

For each interview, the binding:

1. resolves `current.assignment.family_groups.group_id` to a concrete string;
2. looks up that key in `family_polls`;
3. binds the selected state as `current.state`;
4. confines all reads and writes to that state.

Amina and Boris therefore share `family_polls["family-a"]`. Chen and Daria share `family_polls["family-b"]`.

`.by(...)` never creates a missing key. Preflight resolves every assignment and fails before model calls if the map does not contain one of the resulting keys.

8.3 Keys

A state key is a nonempty string identifying one shared world. It can represent a team, committee, classroom, household, or replication:

```

red-team
committee-1
course-2026/section-3

```

The string has no built-in meaning. The research design supplies its meaning. Keys may be added with ordinary dictionary assignment while authoring. They must all exist before preflight freezes the map and execution begins.

Chapter 9

State goes into and comes out of a run

A run treats shared state as serializable data. The user supplies an input snapshot, and `Results` contains both the input and output snapshots:

```
round_1 = (  
    survey  
    .by(people)  
    .by(Model("gemini-2.5-flash"))  
    .run(  
        shared_state=activity_polls.at("weekend-activity"),  
        interview_schedule=Serial(),  
    )  
)  
  
round_1.shared_state.before  
round_1.shared_state.after
```

Both snapshots are immutable. The original `activity_polls` object is not silently rebound to execution infrastructure or changed by a remote worker.

9.1 Continuing with the output of an earlier run

Pass the preceding output snapshot into the next run:

```
round_2 = (  
    follow_up_survey  
    .by(people)  
    .by(Model("gemini-2.5-flash"))  
    .run(  
        shared_state=round_1.shared_state.after.at("weekend-activity"),  
        interview_schedule=Serial(),  
    )  
)
```

This creates an explicit, reproducible chain:

```

activity_polls
  |
  v
round 1 --> round_1.shared_state.after
                |
                v
                round 2 --> round_2.shared_state.after

```

Several **Results** objects can therefore describe successive stages of one study without sharing an implicit mutable store. Each result remains a complete historical record of its own transition.

9.2 Local and remote execution

The same study code works locally and remotely. Shared state cannot, however, be implemented as an ordinary dictionary attached only to the Python object passed to `.run()`.

Local EDSL execution uses a runner architecture. `Jobs.run()` constructs a runner with a job service, centralized storage, a coordinator, a render worker, and an execution-worker pool. The default local runner currently operates in one Python process and can back its service with thread-safe in-memory storage. Jobs and surveys are nevertheless serialized and reconstructed between runner components; execution does not depend on every worker retaining the same original Python object.

Shared state therefore belongs in the runner's centralized service storage. For a default local run, that storage may be in memory. A runner using SQLite, PostgreSQL, or distributed workers can select a different implementation. The researcher-facing **SharedState** and survey syntax remains unchanged.

During remote inference, EDSL creates the remote coordination resource required by its workers. The execution sequence is:

1. EDSL serializes and uploads the **before** snapshot.
2. The run-scoped state service coordinates all reads and writes.
3. EDSL retrieves the final state as the **after** snapshot.
4. Both snapshots are serialized into **Results**.

No JSONL path is required in the study definition. A file-backed event log can remain an internal or explicitly requested runner-storage implementation, not the mechanism by which reconstructed survey objects share state.

A local run can feed a remote run, and a remote run can feed a local run, because the boundary between runs is a snapshot rather than a file path or a live store handle.

Runs that must interact with the same state *at the same time* belong in one execution plan. Snapshot chaining coordinates separate runs sequentially.

Chapter 10

Schedules are typed objects

Execution order determines what participants observe. It is part of the study design, not a string-valued performance option.

10.1 Serial participation

```
from edsl.schedules import Serial

schedule = Serial()
```

Each interview completes before the next begins. In the activity poll, every later participant sees all earlier votes.

10.2 Concurrent independent interviews

```
from edsl.schedules import Concurrent

schedule = Concurrent(max_concurrency=20)
```

Use `Concurrent` when interviews do not depend on writes made by other concurrent interviews.

10.3 Simultaneous rounds

```
from edsl.schedules import ConcurrentRound, RoundEnd, RoundStart

schedule = ConcurrentRound(
    grouping=family_groups,
    snapshot=RoundStart,
    reveal=RoundEnd,
    max_concurrency=20,
)
```

Every participant reads the same round-start snapshot. Writes become visible after the round closes, so model response time does not create an unintended speaking order.

10.4 Ordered participation within concurrent teams

```
from edsl.schedules import GroupedRoundRobin

schedule = GroupedRoundRobin(
    grouping=family_groups,
    rounds=1,
    max_concurrent_groups=10,
)
```

`family_groups` supplies both membership and the stable order within each family. Different families proceed concurrently. The schedule does not need to reconstruct those facts from agent traits or from separate string expressions.

Chapter 11

Extension: one poll per group

The same map can route different groups into isolated polls:

```
from edsl.assignments import AgentGrouping

family_groups = AgentGrouping(
    name="family_groups",
    groups={
        "family-a": ["Amina", "Boris"],
        "family-b": ["Chen", "Daria"],
    },
)

participants = people.with_assignments(family_groups)

family_polls = SharedStateMap({
    "family-a": SharedState(
        votes=SharedRegister(value_options=ACTIVITIES, write_once=True),
    ),
    "family-b": SharedState(
        votes=SharedRegister(value_options=ACTIVITIES, write_once=True),
    ),
})

schedule = GroupedRoundRobin(
    grouping=family_groups,
    rounds=1,
    max_concurrent_groups=10,
)

grouped_results = (
    survey
    .by(participants)
    .by(Model("gemini-2.5-flash"))
    .run(
        shared_state=family_polls.by(
```

```
        current.assignment.family_groups.group_id
    ),
    interview_schedule=schedule,
)
)
```

The grouping is attached to the agents before the survey is composed with them. It does not become part of their persona traits. Its name creates a stable runtime namespace, so another grouping can coexist without colliding with `family_groups`.

The realized map contains two `SharedState` values. Each has a `votes` register with the same structure, but a participant sees only votes from the selected family.

The survey operation was authored against `poll.votes`. Operations serialize their primitive name and type, not the Python identity of that one object. Preflight verifies that every value in `family_polls` has a compatible `votes` primitive before allowing the shared survey to run.

Chapter 12

Primitive reference and question composition

A shared-state primitive owns one kind of coordinated data and the operations allowed to change it. Primitive methods construct survey steps; they do not mutate state while Python builds the survey.

Question composition is typed. A method that expects a numerical answer should accept a `QuestionNumerical`, while a method with a fixed action set should accept a `QuestionMultipleChoice` whose options match that set. EDSL checks these contracts when the operation is created, before a model is called.

The tables below distinguish:

- **method**, the user-facing operation placed in a survey;
- **question input**, the answer shape the method consumes;
- **effect**, the primitive transition requested after that answer.

Every primitive can be observed through `current.state.<name>`. The containing state supplies two additional methods:

State method	Where used	Meaning
<code>state.refresh()</code>	Inside <code>Survey(...)</code>	Construct a deferred refresh step for the bound state
<code>state.read()</code>	During analysis	Immediately inspect one concrete realized state

12.1 General coordination primitives

Primitive	Authoring methods	Natural question composition
<code>SharedRegister</code>	<code>set(key=..., value=...)</code>	Any answer type compatible with its declared value contract; commonly multiple choice, yes/no, numerical, free text, checkbox, rank, budget, or structured data

Primitive	Authoring methods	Natural question composition
SharedLog	<code>append(**fields)</code>	Any collection of typed question answers and runtime references; each field retains its own validated type
SharedCounterMap	<code>tally(question)</code>	<code>QuestionCheckBox</code> whose options exactly match the counter keys
SharedDocument	<code>revise(question, editor=...)</code>	<code>QuestionFreeText</code> containing the proposed full revision or patch
SharedMessageBoard	<code>add(author_question, message_question, reply_to_question=None)</code>	Free-text author, message, and optional reply target
SharedWorkPool	<code>claim_before(question, claimant=...);</code> <code>complete(question, claimant=...)</code>	Claiming occurs before any question; completion can store the validated answer from the work question
SharedSignalSchedule	<code>reveal_before(question, recipient=..., round_number=...)</code>	No answer input; reveals a configured signal before the named question
SharedAgenda	<code>propose(question, proposer=...);</code> <code>vote(question, voter=...)</code>	Free-text proposal; <code>QuestionMatrix</code> with up, neutral, and down for every proposal
SharedNegotiation	<code>record(action_question, amount_question, message_question, ...)</code>	Multiple-choice action, numerical amount, and free-text message
SharedForecast	<code>submit(probability_question, confidence_question, ...)</code>	Two numerical questions with declared probability and confidence bounds
SharedDelphiPanel	<code>submit(estimate_question, confidence_question, rationale_question, ...)</code>	Numerical estimate, numerical confidence, and free-text rationale

For a register, the question's validated answer can be passed explicitly:

```
activity = QuestionMultipleChoice(
    question_name="activity",
    question_text="Which activity do you prefer?",
    question_options=ACTIVITIES,
)

write_vote = poll.votes.set(
    key=current.agent.name,
    value=activity.answer,
)
```

For a domain method with several inputs, passing the questions themselves is a concise typed binding.

The method records their `.answer` references internally:

```
trade = market.orders.trade(
    action_question=action,
    quantity_question=quantity,
)
```

Construction checks that `action` is multiple choice with exactly the market's actions and that `quantity` is numerical. This produces an error at the line where `trade` is created instead of during execution.

12.2 Allocation, matching, and market primitives

Primitive	Authoring methods	Natural question composition
SharedMatchPool	<code>collect(question, participant=...)</code>	<code>QuestionRank</code> whose options exactly match the available items
SharedDeferredAcceptance	<code>collect(question, student=...)</code>	<code>QuestionRank</code> whose options exactly match the institutions
SharedCoalitionPool	<code>request(coalition_question, ...)</code>	Multiple choice whose options exactly match the configured coalitions
SharedBudgetPool	<code>fund(project_question, amount_question, ...)</code>	Multiple-choice project plus numerical amount
SharedResourceBoard	<code>allocate(incident_question, resource_question, ...)</code>	Two multiple-choice questions constrained to configured incidents and resources
SharedAuction	<code>bid(question)</code>	Numerical bid within the mechanism's permitted range
SharedSealedAuction	<code>bid(question, bidder=...)</code>	Numerical bid; results remain hidden until close
SharedDoubleAuction	<code>submit(action_question, price_question, ...)</code>	Multiple-choice buy/sell/hold action plus numerical price
SharedBinaryMarket	<code>trade(action_question, quantity_question, ...); settle(outcome)</code>	Multiple-choice <code>buy_yes/buy_no/hold</code> plus numerical quantity; settlement consumes a literal boolean

12.3 Experimental-game primitives

Primitive	Authoring methods	Natural question composition
SharedUltimatumGame	<code>offer(question); respond(question); act(offer_question, decision_question, ...)</code>	Numerical offer and an accept/reject multiple-choice or yes/no decision
SharedMoneyRequestGame	<code>submit(question)</code>	Numerical request within the configured interval

Primitive	Authoring methods	Natural question composition
SharedMatrixGame	<code>submit(question, player=..., seat=...)</code>	Multiple choice whose options exactly match the available actions
SharedRepeatedMatrixGame	<code>submit(question, player=..., seat=..., round_number=...)</code>	The same action question, repeated under a typed round schedule
SharedDictatorGame	<code>allocate(question, player=...)</code>	Numerical allocation bounded by the endowment
SharedTrustGame	<code>send(question, player=...); return_funds(question, player=...)</code>	Numerical transfers bounded by funds available at each stage
SharedBeautyContest	<code>submit(question, player=...)</code>	Numerical guess within the declared contest range
SharedCommonPoolGame	<code>extract(question, player=...)</code>	Numerical extraction bounded by the configured individual maximum
SharedCentipedeGame	<code>move(question, player=..., node=...)</code>	Multiple-choice take/pass action
SharedMarketEntryGame	<code>submit(question, player=...)</code>	Yes/no or multiple-choice enter/stay-out decision
SharedBilateralTrade	<code>offer(question, seller=...); respond(question, buyer=...)</code>	Numerical offer and accept/reject decision
SharedSignalingGame	<code>signal(question, sender=...); decide(question, employer=...)</code>	Configured signal choice followed by a configured response choice
SharedNashDemandGame	<code>demand(question, player=..., seat=...)</code>	Numerical demand bounded by the available pie
SharedVotingGame	<code>vote(question, voter=...)</code>	Multiple choice whose options exactly match the candidates
SharedCheapTalkGame	<code>message(question, sender=...); act(question, receiver=...)</code>	Free-text or configured message followed by a configured action choice
SharedPrincipalAgentGame	<code>contract(question, principal=...); effort(question, worker=...)</code>	Structured contract choice followed by a configured effort choice

12.4 Configured primitives

When a study does not need a named domain class, a configured primitive can declare its fields, operations, and transition rules from simpler data:

```
from edsl.sharedstate.configured_game import (
    Action,
```

```

    ConfiguredSharedGame,
    Equals,
    Field,
    Ref,
    Settlement,
)

allocation = ConfiguredSharedGame(
    fields={
        "amount": Field.number(minimum=0, maximum=100),
        "decision": Field.choice(("accept", "reject")),
    },
    actions={
        "offer": Action(actor="proposer", writes="amount"),
        "respond": Action(
            actor="responder",
            writes="decision",
            requires=("amount",),
        ),
    },
    terminal_when_set="decision",
    settlement=Settlement(
        when=Equals(Ref("decision"), "accept"),
        payoffs={},
    ),
)

```

Its `.bind(...)` method associates a named operation with one or more questions. Binding validates answer shapes and option domains immediately. The resulting operation is still a typed, serializable survey step and follows the same advisory-write contract as built-in primitives.

12.5 Compatibility is checked at creation

Primitive methods should reject incompatible compositions immediately:

```

QuestionCompatibilityError: SharedBinaryMarket.trade quantity expects a
QuestionNumerical; 'quantity' is QuestionFreeText.

```

```

QuestionOptionsError: SharedVotingGame.vote expected options
['bike ride', 'sailing', 'hike', 'beach day']; question 'activity' also contains
'movie'.

```

```

QuestionBoundsError: SharedDictatorGame.allocate allows values from 0 to 100;
question 'allocation' allows values through 150.

```

Type compatibility does not establish scientific validity. EDSL can verify that an auction bid is numerical and within bounds; the researcher still decides whether the prompt, information treatment, incentives, and schedule represent the intended experiment.

Chapter 13

Inspecting a completed run

The completed run exposes the collection before and after execution:

```
before = results.shared_state.before
after = results.shared_state.after

instance(after, SharedStateMap) # True
after.realized_keys()           # ["weekend-activity"]
```

Indexing returns one concrete `SharedState`:

```
poll = after["weekend-activity"]

instance(poll, SharedState) # True
poll.key                    # "weekend-activity"
poll.read()
poll.render_markdown()
```

The result also contains the events produced during the transition:

```
results.shared_state.events
results.shared_state.events.for_target("votes")
```

Each event identifies its state key, primitive target, operation, and ordering metadata.

Every successful initial read or explicit `.refresh()` is a first-class Results-level state-read record with a stable ID:

```
read = results.state_reads["state-read-017"]

read.snapshot_id
read.interview_id
read.state_key
read.state_version
read.survey_step
read.implicit
read.schedule_view
```

State snapshots are deduplicated and stored once:

```
snapshot = results.state_snapshots[read.snapshot_id]
```

Each ordinary result row carries the ID of the state view active when its question was rendered:

```
results.select(  
    "answer.reaction",  
    "prompt.reaction",  
    "state_read_id.reaction",  
)
```

Questions between two reads share the same `state_read_id`. This includes questions that do not interpolate a state field: the ID records the complete interview context in force at that point. A question in a study with no shared state has a null state-read ID.

Failed read attempts are execution errors, not successful read records. They are retained in task diagnostics and cannot be referenced as the state view for a rendered question.

The distinction between authoring and inspection is now visible:

Before the run:	activity_polls	# supplied state value
After the run:	results.shared_state.before	# immutable input snapshot
	results.shared_state.after	# immutable output snapshot
	results.shared_state.events	# transition audit trail
	results.state_reads	# observation records by ID
	results.state_snapshots	# deduplicated snapshot values

Chapter 14

Closing states during execution

A state may be open or closed. Closing prevents later writes and lets primitives perform any final calculation. The schedule declares when a state is complete, including the policy for failed or missing participants. EDSL closes it as part of execution when that condition holds.

The resulting closed status appears in the **after** snapshot. Snapshots inside **Results** are immutable; inspecting one cannot change the historical record.

Chapter 15

Serialization and remote execution

The following objects serialize with the job:

- `SharedStateMap`, its `SharedState` values, and every primitive configuration;
- the fixed or dynamic state binding;
- named `AgentGrouping` objects attached to the `AgentList`;
- `current` reference expressions;
- `Stem` rendering plans and read dependencies;
- schedule objects;
- the input shared-state snapshot.

A remote worker reconstructs the map and its values without importing the local example file. It resolves the state binding for each interview and uses run-scoped remote coordination internally.

The resulting `Results` serialization contains:

- `shared_state.before`, the immutable input snapshot;
- `shared_state.after`, the immutable output snapshot;
- `shared_state.binding`, the fixed or dynamic routing used by the run;
- `shared_state.events`, the ordered transition audit trail.
- `state_reads`, the Results-level read records indexed by stable ID;
- `state_snapshots`, the deduplicated values referenced by those records;
- `state_read_id` on result rows and rendered-prompt provenance.

`before` and `after` refer into the same serialized snapshot collection. They are convenient names for the run boundaries; the read records preserve every state view that affected execution between those boundaries.

No local path, live Python object, or durable remote-store reference is needed to continue the study. A later run can use the serialized `after` snapshot as its input.

Arbitrary Python callbacks are not part of the normal path. A custom callable that lacks a declarative representation makes a job local-only and fails remote preflight.

Chapter 16

Validation before execution

The interface detects structural errors before model calls begin:

```
PrimitiveNameError: 'read' is reserved and cannot be used as a primitive name.
```

```
ReferenceError: current.assignment.family_groups.grop_id is not declared; did you mean 'group_id'?
```

```
StateBindingError: family_polls.by(...) must resolve to a nonempty string, but agent 'Amina' has no assignment named 'family_groups'.
```

```
StateRoutingError: family_groups assigns agent 'Amina' to 'family-c', which is not a key in family_polls. Available keys: family-a, family-b.
```

```
SurveyOrderError: votes.set reads activity.answer before question 'activity' runs.
```

```
GroupingError: agent 'Amina' appears twice in family_groups['family-a'].
```

```
VisibilityError: question 'activity' is not authorized to read private field votes.internal_metadata.
```

```
StateSnapshotError: shared_state must be a SharedStateMap or StateBinding; results.shared_state.after is an immutable SharedStateMap and may be reused.
```

The system cannot infer every scientific decision. Authors still specify what happens when an interview fails, when concurrent writes become visible, how missing participants affect completion, and whether retries reuse previous answers.

Chapter 17

Mental checklist

Before running a coordinated study, an author should be able to answer:

1. Which named primitives does each shared world contain?
2. Does every interview use one fixed key, or is the map routed with `.by(...)`?
3. What concrete entity does each key represent?
4. Which questions read shared state through `Stem`?
5. Which operations occur before a question or after an answer?
6. What schedule determines ordering and visibility?
7. When does each realized state close?
8. Can the map, binding, and schedule serialize for remote execution?
9. If this follows an earlier run, is its `after` snapshot the intended input?
10. Does the completed `Results` object preserve the `before`, `after`, and transition events?

When those answers are visible in the Python program, the simulation is easier to write, inspect, reproduce, and review as a scientific design.